# Practice Guide 3: Breaking the Filters (Counter Examples and Useful Examples)

Aprendizagem Aplicada à Segurança

Mário Antunes

October 9, 2026

## Objective

You play the **attacker** against the filters of the previous guides, at two levels, and then you defend.

- **Level 1, counter example:** find a pattern that bypasses **Naive Bayes**, **logistic regression** and an **embedding model**. You need not keep the message working; you need to *name the blind spot*.
- **Level 2, useful example:** find a rewrite that still works as a scam: the **payload** (phone numbers and short codes, links, amounts, call to action) is intact, the **meaning** is kept (an independent embedder says so), and it **looks normal** to a reader. This is what a scam campaign could reuse.
- **Defend:** normalise the input, retrain on the attack, and measure what the attacker does next.

**Lab use only.** You attack the models you trained yourself, on the public SMS corpus. Before writing code, state the **threat model**.

| Axis | This lab |
|---|---|
| **Goal** | a spam message is classified as ham (*evasion*) |
| **Capability** | append words, replace words, insert characters; level 2: the payload must stay intact |
| **Knowledge** | white-box (weights) or black-box (a score per message: each score is a *query*) |
| **Constraint** | level 2: payload, meaning and a normal look |

- **Notebook to complete:** `guide_02_03.ipynb`. **Time:** about 2 hours (suggested: A 10 min, B 30, C 20, D 35, E 25). **Prerequisites:** Guides 1 and 2 and the lecture of class 4.

- **Requirements:** `make venv`. You also need the llama-swap stack (`make llama-up`, see Guide 2): the `minilm` model is the *judge of meaning* in part D, `embgemma` is an optional third victim (`AAS_SLM=skip` leaves the EmbeddingGemma victim out). Set `AAS_W2V=skip` to skip the Google News download in part D (no synonym operator then).

# Background

**Additive scores.** For Multinomial Naive Bayes the log-odds of a message are exactly $z = \log \frac{P(\text{spam})}{P(\text{ham})} + \sum_w c_w \, \text{llr}_w$, with $\text{llr}_w = \log \frac{P(w|\text{spam})}{P(w|\text{ham})}$: every word adds its own evidence, so the cheapest attack is a *sort*: add the words with the most negative llr.

**Black-box search.** The attacker cannot read the model; they *evaluate* it. Greedy search: append every candidate word, keep the one that lowers the score most, stop when the score is negative. With $m$ candidates and $s$ steps the cost is about $1 + m \cdot s$ **queries**.

**Dilution.** TF-IDF vectors are L2-normalised and embeddings are averaged over tokens: benign text makes the spam a smaller share of the input.

**Level 2.** Minimise the change subject to: the filter says *ham*, the **payload** is unchanged, the **visible** text stays close to the original (cosine of an independent embedder $\geq \tau$), and at most $B$ words differ *for a human* (Cyrillic look-alike letters and zero-width spaces are invisible, so they cost nothing). Allowed edits look normal: invisible characters, real benign sentences, near-synonyms, paraphrases; junk words are not.

**Defences and the arms race.** *Normalisation* (NFKC, remove invisible characters, map look-alikes back) closes the invisible attacks. *Adversarial training* covers the attacks it has seen; the attacker adapts. *Removing the score* (label-only API) and *limiting queries* raise the attacker's price. Each defence has a cost and an answer.

# Tasks

| Part | What you do | Check |
|---|---|---|
| **A** | Threat model, victims and the public candidate words (given) | |
| **B** | **NB**: reproduce the score from the log-odds, closed-form attack; implement the **greedy black-box search**; ASR curves | `assert` on the score, Questions B1 to B3 |
| **C** | **Dilution**; **transfer** of the crafted messages to the other victims | table, Questions C1, C2 |

| Part | What you do | Check |
|---|---|---|
| **D** | Implement the **payload check** with regular expressions; run a **constrained search** (camouflage, synonyms, invisible characters) and measure the **useful ASR** | `assert` on the payload check, Questions D1 to D3 |
| **E** | Implement **normalisation**; **adversarial training** and re-attack | `assert`, Questions E1 to E3 |

**Tips.** Every attack is an optimisation problem: gradients when the model is differentiable and known, derivative-free search when it is not. Count queries. Print the adversarial messages: what a model accepts and what a person accepts are different questions.

# Questions

1. **(B1)** What are `lt` and `gt`, why are they so "hammy", and what does that say about the data and about the attacker's cost?
2. **(B2)** At budget 0 some messages already evade. What are they, and how does this change the meaning of "attack success rate"?
3. **(B3)** Compare the white-box closed form with the black-box curve for Naive Bayes. Which knowledge is worth more, and how many queries did the black-box attacker pay?
4. **(C1)** Which victim resists dilution best? Explain with the way each model turns a message into a score.
5. **(C2)** Do the messages crafted against one victim also evade the others? Why (not)?
6. **(D1)** Compare the useful ASR with the level 1 evasion of Part B. What is the *price of usefulness*, and which constraint costs most?
7. **(D2)** Look at the example message: what did the attacker change, and why would a recipient not notice?
8. **(D3)** Which scam family would you expect to be easiest to disguise, and why? Test it on your messages (`sl.spam_families`).
9. **(E1)** What does normalisation cost, and which of the attacks of Parts B to D does it not touch?
10. **(E2)** Report the clean F1 and the evasion before and after adversarial training. Why does the attack not go to zero, and what would you try next as the attacker?
11. **(E3)** Name one defence that takes away the attacker's *feedback* instead of hardening the model, and the attacker's answer to it.
12. **(Ethics)** The same code that measures robustness also helps a scammer. Which parts of what you built would you *not* publish, and why?

| Victim | Level 1 evasion at 12 words | Level 2 useful ASR | after normalisation (homoglyph attack, recall) |
|---|---|---|---|
| Naive Bayes (counts) | | | |
| TF-IDF + logistic regression | | | |
| EmbeddingGemma 2 + LR (if available) | | | |

## Optional challenges

- **Universal suffix:** find a single 8-word suffix, on training spam only, that evades most of the test spam (`al.universal_trigger`). Which words does it contain?
- **Poisoning:** send spam with a rare trigger token, labelled *not spam*, into the training set (notebook 10, part A). Which model is more vulnerable, and why? Try a label-noise filter.
- **Judge:** use EmbeddingGemma instead of MiniLM as the judge of meaning (`al.make_judge("embgemma")`). Does the useful ASR change?
- **Paraphrase:** use the paraphrases of `datasets/spam_paraphrases.json` (made once by the local Gemma 4 E2B model with the payload protected by placeholders) as one more operator (`al.op_paraphrase`).

## Reference solution

The solved notebook is `solution/guide_02_03.ipynb`.